

操作系统

Principles of Operating System

北京大学计算机科学技术系 陈向群

Department of computer science and
Technology, Peking University

2026 Spring

I/O管理

- ▶ I/O管理概述
- ▶ I/O硬件组成
- ▶ I/O软件组成
- ▶ I/O管理相关技术
- ▶ I/O性能问题

问题

- ▶ 如何感知设备的状态并管理设备？
- ▶ 管理设备时，如何解决各类设备差异性大的问题？
- ▶ 为什么要对设备建立抽象，统一对设备的访问接口？
- ▶ 如何提高CPU 与设备的访问性能？
- ▶ 为什么引入缓冲技术？
- ▶ 一个I/O请求处理流程是怎样的？

I/O管理概述

1. I/O的特点

- ▶ I/O的性能经常是系统性能的瓶颈
- ▶ 操作系统庞大复杂的原因之一：
资源多、杂，并发，均来自I/O
 - ▶ 速度差异很大
 - ▶ 应用
 - ▶ 控制接口的复杂性
 - ▶ 传送单位
 - ▶ 数据表示
 - ▶ 错误条件
- ▶ 与其他功能联系密切，
特别是文件系统

2.设备的发展

- ▶ 简单设备

CPU 可通过I/O 接口直接控制I/O 设备

- ▶ 多设备

CPU 与I/O 设备之间增加了一层I/O 控制器和总线BUS

- ▶ 支持中断的设备——提高CPU 利用率

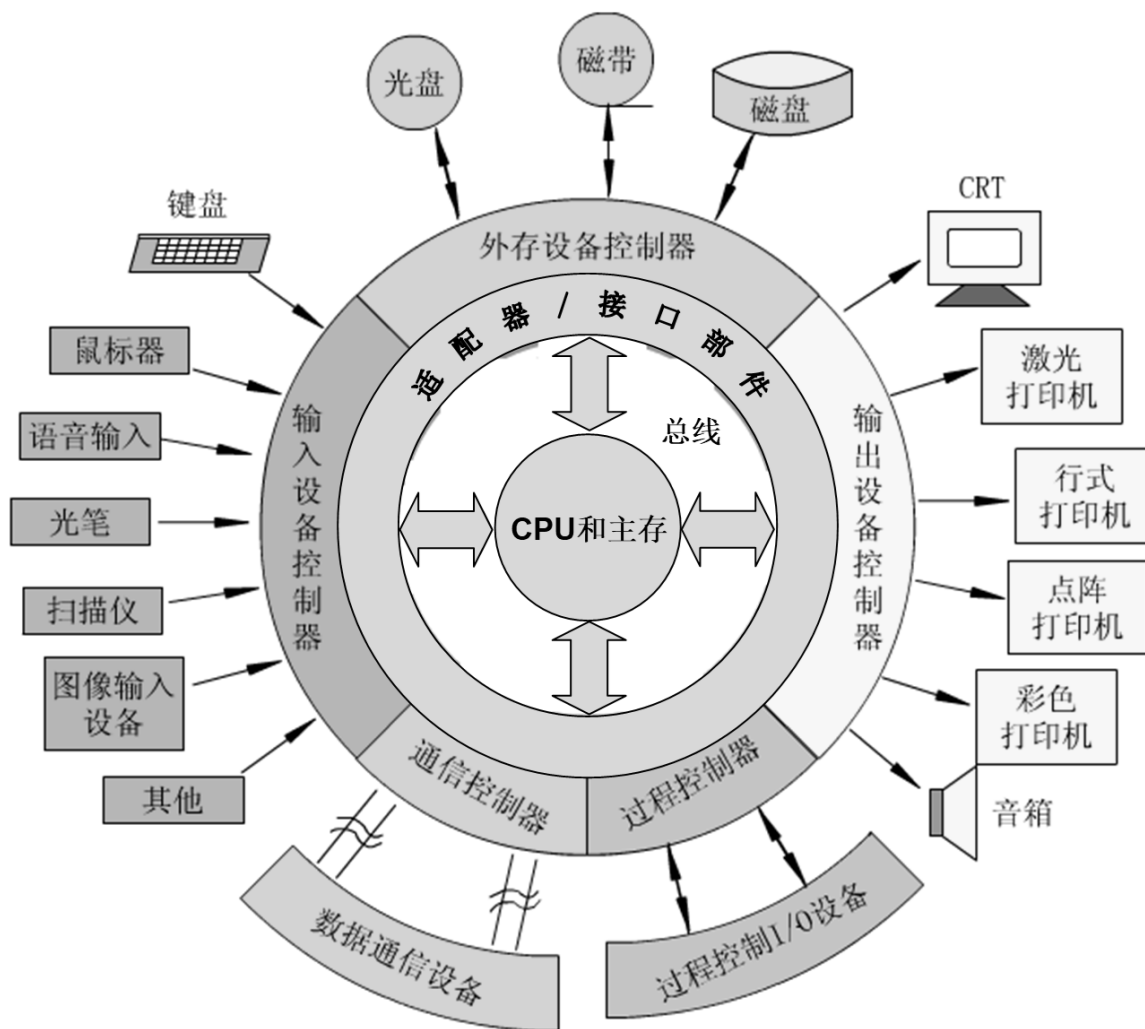
- ▶ 高吞吐量设备——支持DMA

- ▶ 各种其他设备

GPU、声卡、智能网卡、RDMA

3.计算机I/O系统结构

连接方式：直连、（设备/中断）控制器、总线、分布式



► CPU与设备的连接

✓ 设备控制器

- 1.CPU和I/O设备间的接口
- 2.向CPU提供特殊指令和寄存器
- 3.内存地址或端口号

4.设备的分类——按数据特征分

► 字符设备

以字节为单位存储、传输信息（GPIO、键盘、鼠标、串口）
传输速率低、不可寻址

I/O命令：get(); put(); 通常使用文件访问接口和语义

► 块设备

以数据块为单位存储、传输信息（磁盘、磁带、光驱）
传输速率较高、可寻址（随机读写）

I/O命令：原始I/O或文件系统接口；内存映射文件访问

► 网络设备

格式化报文交换（以太网、无线、蓝牙）

I/O命令：send/receive 网络报文；通过网络接口支持多种网络协议

设备的分类——从资源分配角度

▶ 独占设备

在一段时间内只能有一个进程使用的设备，一般为低速I/O设备（如打印机，磁带等）

▶ 共享设备

在一段时间内可有多个进程共同使用的设备，多个进程以交叉的方式来使用设备，其资源利用率高（如硬盘）

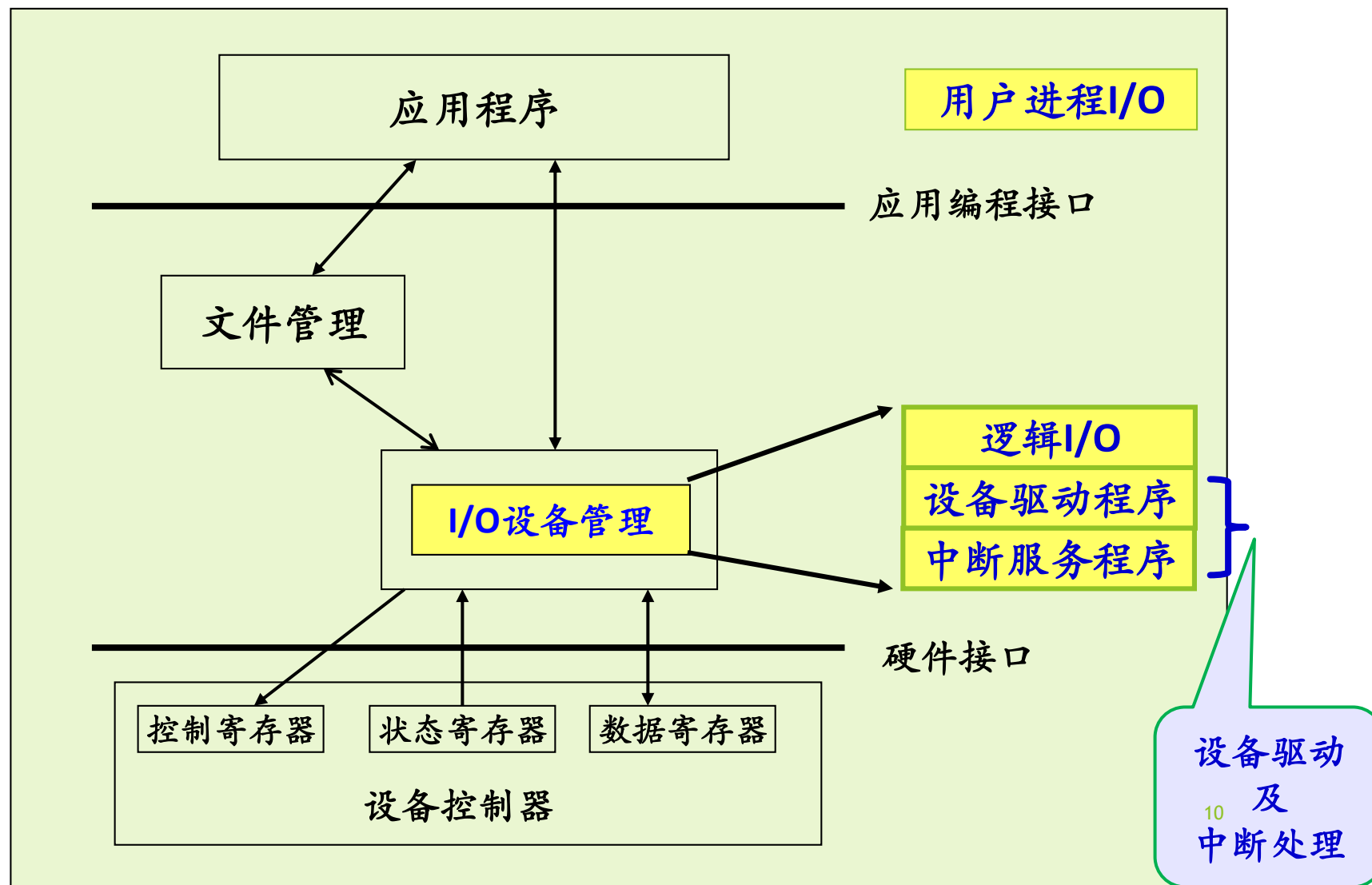
▶ 虚设备

在一类设备上模拟另一类设备，常用共享设备模拟独占设备，用高速设备模拟低速设备；被模拟的设备称为虚设备

目的：将慢速的独占设备改造成多个用户可共享的设备，提高设备的利用率

实例：SPOOLing技术，利用虚设备技术——用硬盘模拟输入输出设备

5. I/O管理示意图



6. I/O管理的目标和任务 (1/3)

(1) 按照用户的请求，控制设备的各种操作，完成I/O设备与内存之间的数据交换，最终完成用户的I/O请求

- ▶ 设备分配与回收

- ▶ 记录设备的状态

- ▶ 根据用户的请求和设备的类型，采用一定的分配算法，选择一条数据通路

- ▶ 执行设备驱动程序，实现真正的I/O操作

- ▶ 设备中断处理：处理外部设备的中断

- ▶ 缓冲区管理：管理I/O缓冲区

设备管理的目标和任务 (2/3)

(2) 建立方便、统一的独立于设备的接口

- ◎ 方便性：向用户提供使用外部设备的方便接口，使用户编程时不考虑设备的复杂物理特性
- ◎ 统一性：对不同的设备采取统一的操作方式，在用户程序中使用的是逻辑设备
- ◎ 逻辑设备与物理设备、屏蔽硬件细节（设备的物理细节，错误处理，不同I/O的差异性）

通用性

种类繁多、结构各异
→ 设计简单、避免错误
→ 采用统一的方式处理所有设备

设备管理的目标和任务(3/3)

(3) 充分利用各种技术（通道，中断，缓冲，异步 I/O 等）提高 CPU 与设备、设备与设备之间的并行工作能力，充分利用资源，提高资源利用率

- ▶ 并行性

- ▶ 均衡性（使设备充分忙碌）

(4) 保护

设备传送或管理的数据应该是安全的、不被破坏的、保密的

性能

CPU 与 I/O 的速度差别大

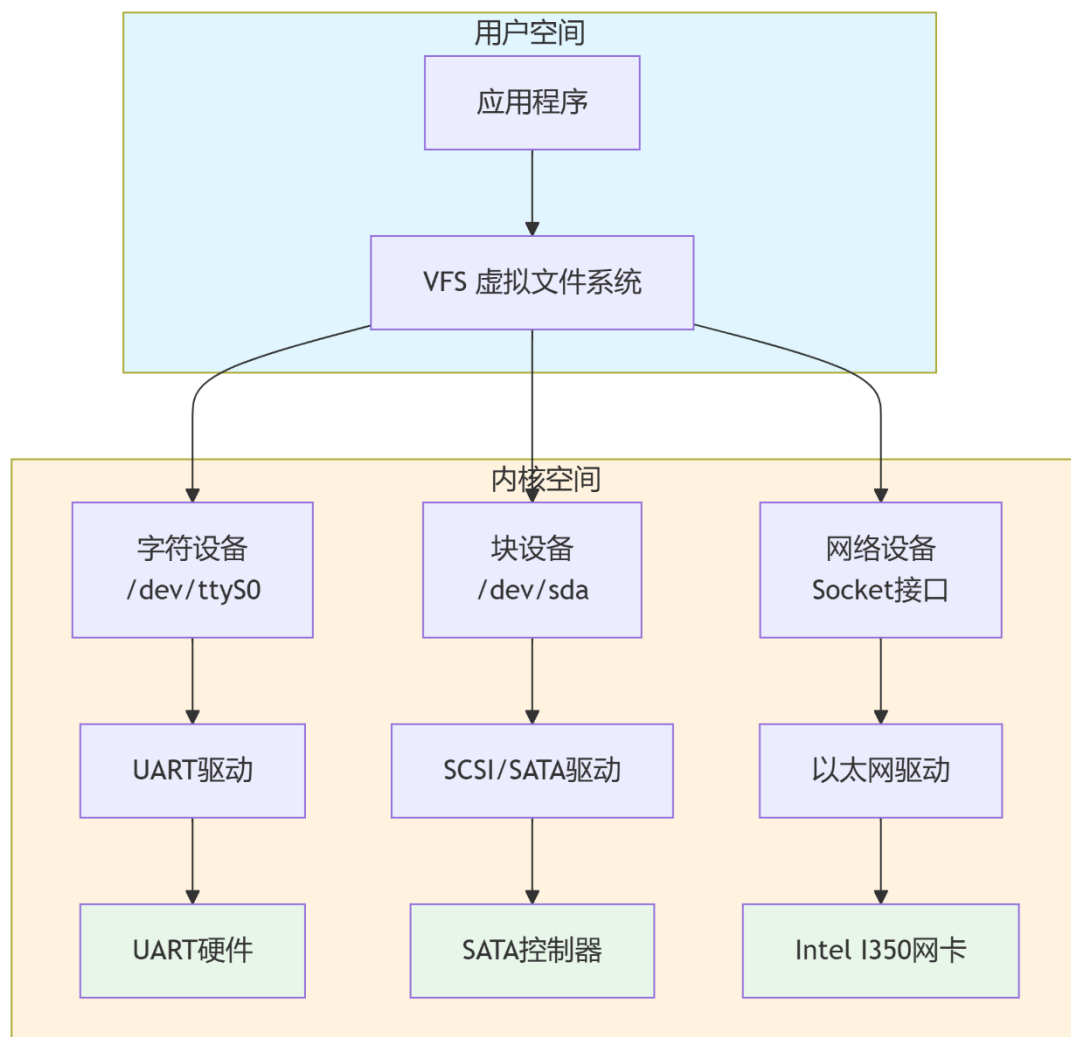
→ 减少由于速度差异造成的整体性能开销

→ 尽量使两者交叠运行

设备驱动模型

硬件差异 vs. 应用统一

解决方案：引入中间层VFS



设备驱动模型的任务
提供统一接口，封装底层差异

struct file
struct inode
struct dentry

函数指针表实现多态

I/O硬件组成

1.I/O设备组成

► I/O设备一般由机械和电子两部分组成

- (1) 机械部分是设备本身（物理装置）
- (2) 电子部分又称设备控制器(或适配器)
 - （端口）地址译码
 - 按照主机与设备之间约定的格式和过程接受计算机发来的数据和控制信号 或 向主机发送数据和状态信号
 - 将计算机的数字信号转换成机械部分能识别的模拟信号，或反之
 - 实现设备内部硬件缓冲、数据加工等提高性能或增强功能

2.设备接口——控制器的作用

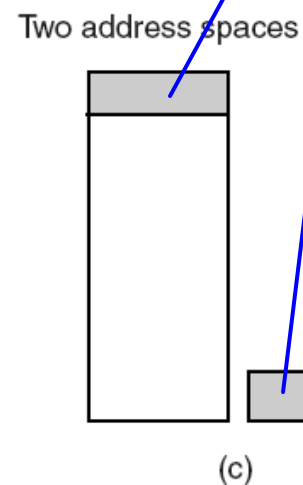
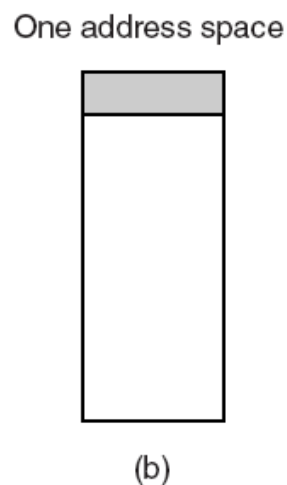
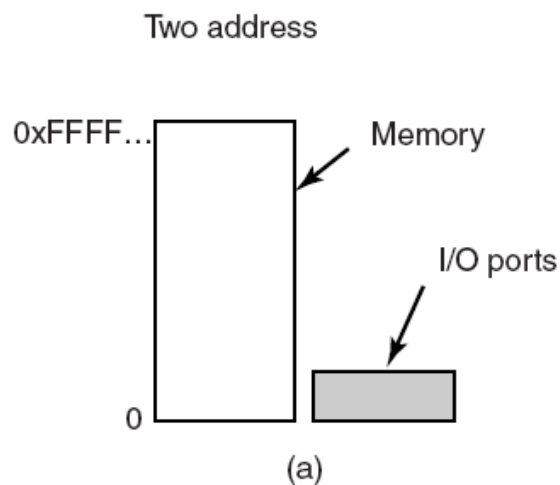
- ▶ 操作系统将命令写入控制器的接口寄存器（或接口缓冲区）中，以实现输入 / 输出，并从接口寄存器读取状态信息或结果信息
- ▶ 当控制器接受一条命令后，可**独立于CPU**完成指定操作，CPU可以另外执行其他计算；命令完成时，控制器产生一个中断，CPU响应中断，控制转给操作系统；通过读控制器寄存器中的信息，获得操作结果和设备状态
- ▶ 控制器与设备之间的接口常常是一个低级接口
- ▶ 控制器的任务：把串行的位流转换为字节块，并进行必要的错误修正：首先，控制器按位进行组装，然后存入控制器内部的缓冲区中形成以字节为单位的块；在对块验证检查和并证明无错误时，再将它复制到内存中

3. I/O端口地址

- ▶ I/O端口地址：接口电路中每个寄存器具有的、唯一的地址，是个整数，亦称为I/O地址
- ▶ 所有I/O端口地址形成I/O端口空间(受到保护)

I/O指令形式与I/O地址是相互关联的，主要有两种形式：

- ▶ 内存映射编址（内存映射I/O模式）
- ▶ I/O独立编址（I/O专用指令）



内存映射I/O
的数据缓冲区

独立I/O端口
的控制寄存器

I/O独立编址

- ▶ 分配给系统中所有端口的地址空间完全独立，与内存地址空间无关
- ▶ 使用专门的I/O指令对端口进行操作
- ▶ 优点
 - ▶ 外设不占用内存的地址空间
 - ▶ 编程时，易于区分是对内存操作还是对I/O端口操作
- ▶ 缺点：I/O端口操作的指令类型少，操作不灵活
- ▶ 例子：8086/8088，分配给I/O端口的地址空间64K，0000H~0FFFFH，只能用in和out指令对其进行读写操作

I/O地址范围	设备
000 – 00F	DMA控制器
020 – 021	中断控制器
040 – 043	定时器
060 – 063	键盘
200 – 20F	游戏控制器
2F8 – 2FF	辅助串口
320 – 32F	硬盘控制器
378 – 37F	并口
3D0 – 3DF	图像控制器
3F0 – 3F7	磁盘驱动控制器
3F8 – 3FF	主串口

内存映射编址

- ▶ 分配给系统中所有端口的地址空间与内存的地址空间统一编址
- ▶ 把I/O端口看作一个存储单元，对I/O的读写操作等同于对内存的操作
- ▶ 优点
 - ▶ 凡是可对内存操作的指令都可对I/O端口操作
 - ▶ 不需要专门的I/O指令
 - ▶ I/O端口可占有较大的地址空间
- ▶ 缺点
 - ▶ 占用内存空间

两者比较 (1/2)

内存映射
I/O的优点

- ▶ I/O设备驱动程序：C语言 or 汇编语言？

- ▶ 不需要特殊的保护机制来阻止用户进程执行I/O操作

操作系统必须要做的事情：避免把包含控制寄存器的那部分地址空间放入任何用户的虚拟地址空间之中

- ▶ 可以引用内存的每一条指令也可以引用控制寄存器

例如，如果指令TEST可以测试一个内存字是否为0，那么它也可以用来测试一个控制寄存器是否为0

```
LOOP: TEST PORT-4    //检测端口4是否为0
      BEQ READY      //如果为0，转向READY
      BRANCH LOOP    //否则，继续测试
READY:
```

两者比较 (2/2)

内存映射
I/O的缺点

- ▶ 对一个设备控制寄存器不能进行高速缓存
- ▶ 考虑以下汇编代码循环，第一次引用PORT_4将导致它被高速缓存，随后的引用将只从高速缓存中取值并且不会再查询设备，之后当设备最终变为就绪时，软件将没有办法发现这一点，结果循环将永远进行下去
- ▶ 为避免这一情形，硬件必须针对每个页面具备选择性禁用高速缓存的能力，操作系统必须管理选择性高速缓存，所以这一特性为硬件和操作系统两者增添了额外的复杂性

```
LOOP: TEST PORT-4    //检测端口4是否为0
      BEQ READY      //如果为0，转向READY
      BRANCH LOOP    //否则，继续测试
READY:
```

4.I/O控制方式 (1/3)

CPU与设备控制
器的数据传输

(1) 可编程I/O (轮询/查询)

由CPU代表进程给I/O模块发I/O命令，进程进入忙等待，直到操作完成才继续执行

(2) 中断驱动I/O

目的：减少设备驱动程序不断地询问控制器状态寄存器的开销

实现：I/O操作结束后，由设备控制器主动通知设备驱动程序

(3) DMA

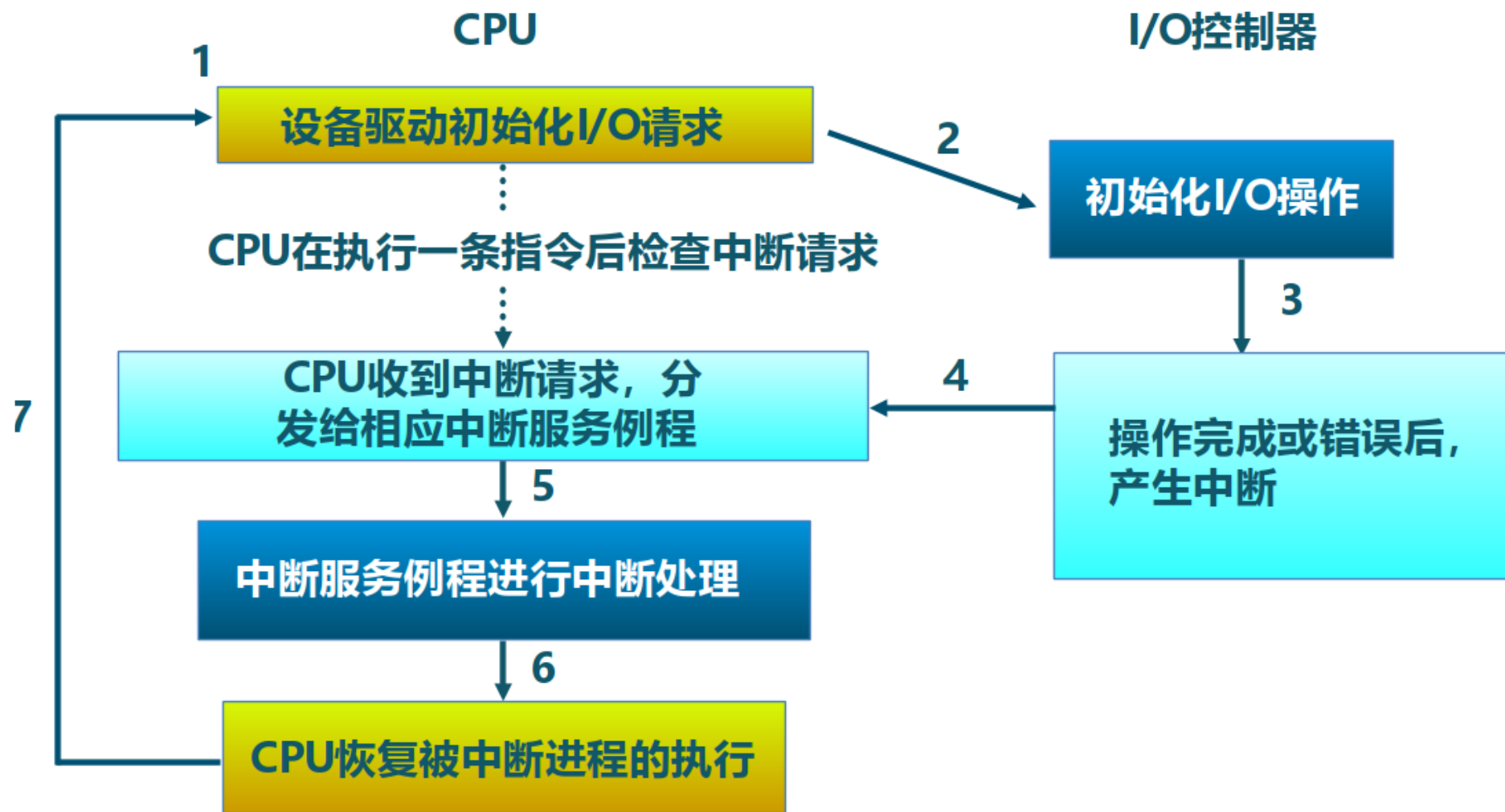
	无中断	使用中断
通过CPU实现I/O-内存间的传送	可编程I/O	中断驱动I/O
I/O-内存间直接传送		直接存储器访问 (DMA)

程序控制I/O(PIO, Programmed I/O)

- ▶ Port-mapped 的PIO(PMIO): 通过CPU 的in/out 指令
- ▶ Memory-mapped 的PIO(MMIO): 通过load/store 传输所有数据
- ▶ 硬件简单, 编程容易
- ▶ 消耗的CPU 时间和数据量成正比
- ▶ 适用于简单的、小型的设备I/O
- ▶ I/O 设备通知CPU: PIO 方式的轮询

中断传输方式

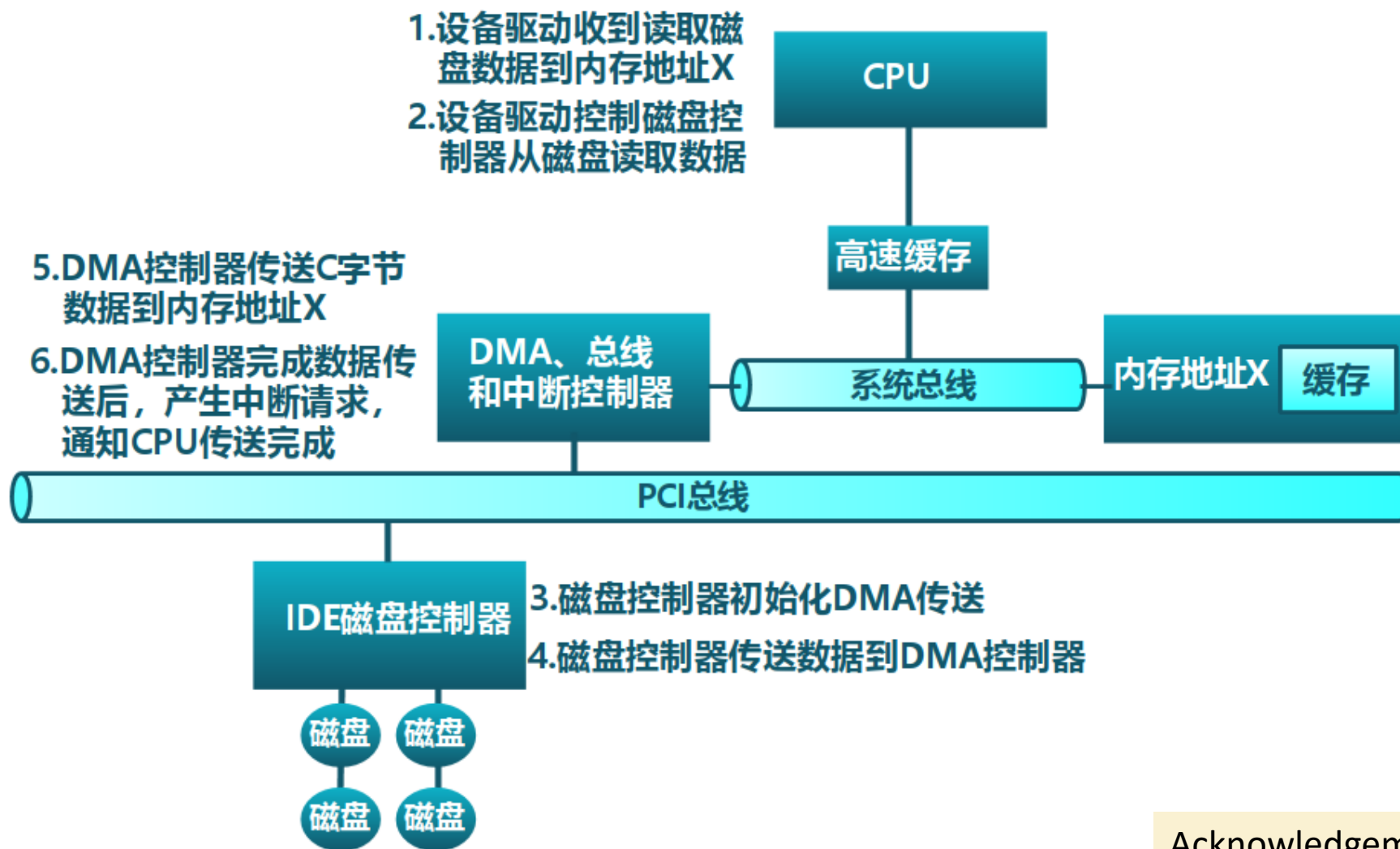
- ▶ I/O 设备向CPU 发送中断请求信号：将数据准备好等状态信息通知CPU
- ▶ 支持中断的设备类型和中断类型逐步增加
- ▶ 需要同时设置CPU 和中断控制器
- ▶ 编程比较麻烦
- ▶ CPU 利用率高
- ▶ 适用于比较复杂的I/O 设备
- ▶ I/O 设备通知CPU：中断方式的提醒



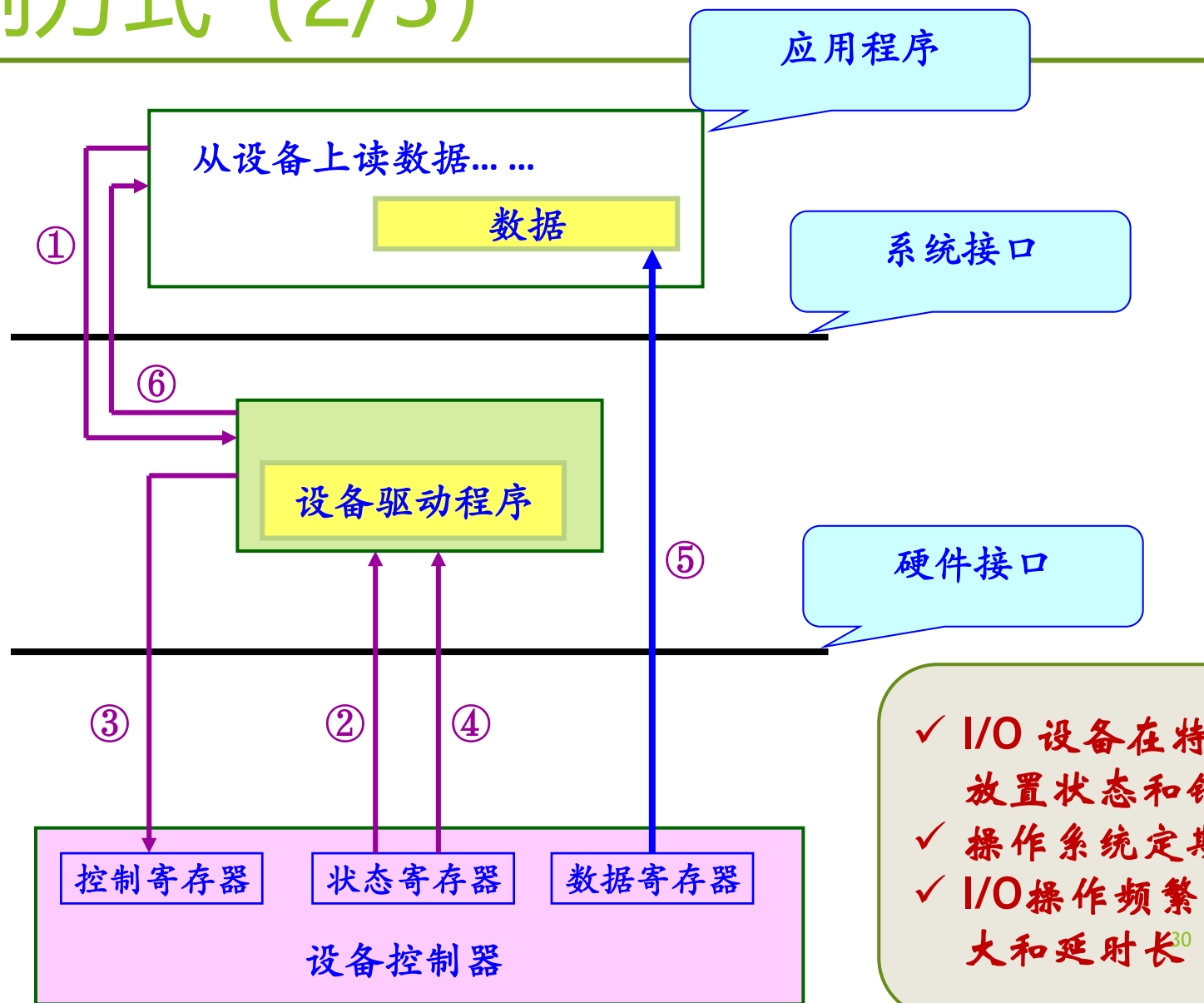
DMA 传输方式

- ▶ 设备控制器可直接访问系统总线
- ▶ 控制器直接与内存互相传输数据
- ▶ 需要同时设置CPU、DMA 控制器和中断控制器
- ▶ 编程比较麻烦，需要CPU 参与设置
- ▶ 设备传输数据不影响CPU
- ▶ 适用于高吞吐量I/O 设备

读取磁盘数据的例子

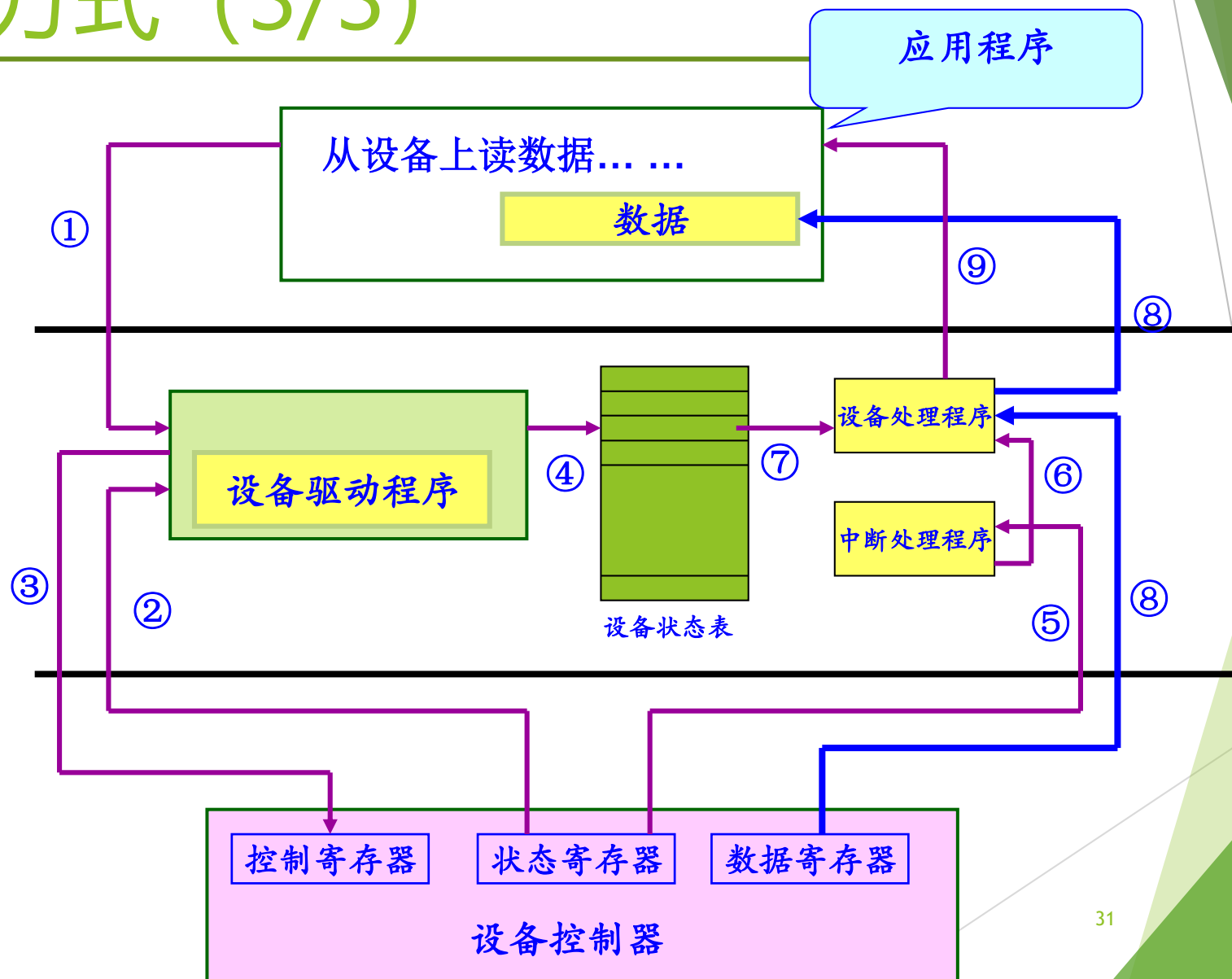


I/O控制方式 (2/3)



- ✓ I/O 设备在特定的状态寄存器中放置状态和错误信息
- ✓ 操作系统定期检测状态寄存器
- ✓ I/O操作频繁或不可预测时，开销大和延时长³⁰

I/O控制方式 (3/3)



I/O部件的演化

性能/CPU
与I/O分离

1. CPU直接控制外围设备
2. 增加了控制器或I/O部件，CPU使用非中断的可编程I/O（CPU开始从外部设备接口的具体细节中分离出来）
3. 与2相同，但采用了中断方式。CPU无需花费等待执行一次I/O操作所需的时间，因而提高了效率
4. I/O部件通过DMA直接控制存储器，可以在没有CPU参与的情况下，从内存中移出或者往内存中移入一块数据，仅仅在传送开始和结束时需要CPU干预
5. I/O部件增强为一个单独的处理器，有专门为I/O设计的指令集；CPU指导I/O处理器执行内存中的一个I/O程序。I/O处理器在没有CPU干涉的情况下取指令并执行这些指令
6. I/O部件有自己的局部存储器(其本身就是一台计算机)

使用这种体系结构可以控制许多I/O设备，并且使需要CPU参与程度降到最小（通常用于控制与交互终端的通信，I/O处理器负责大多数控制终端的任务）

I/O软件的组成

I/O软件设计

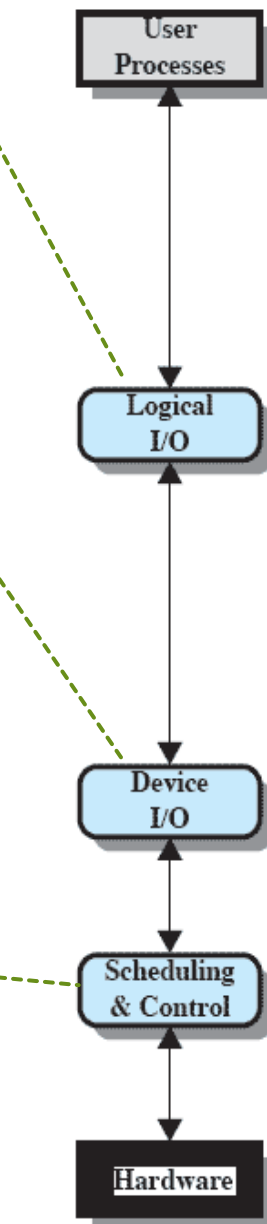
分层的设计思想

- ▶ 把I/O软件组织成多个层次
- ▶ 每一层都执行操作系统所需要的功能的一个相关子集，它依赖于更低一层所执行的更原始的功能，从而可以隐藏这些功能的细节；同时，它又给高一层提供服务
- ▶ 较低层考虑硬件的特性，并向较高层软件提供接口
- ▶ 较高层不依赖于硬件，并向用户提供一个友好的、清晰的、简单的、功能更强的接口

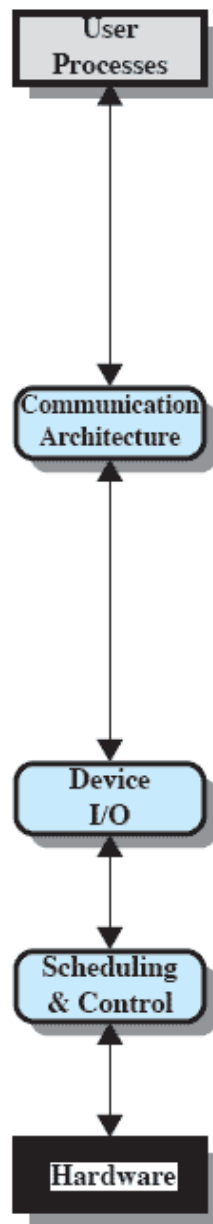
把设备当作一个逻辑资源来处理，不关心实际控制设备的细节

请求的操作和数据被转换成适当的I/O指令序列、通道命令和控制器指令

I/O操作的排队、调度；处理中断，收集并报告I/O状态



(a) Local peripheral device



(b) Communications port



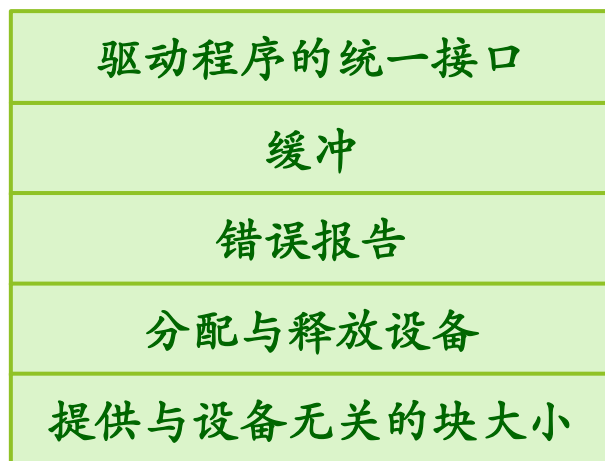
(c) File system

符号文件名→标识符→文件描述符表或索引表→访问文件

处理文件的逻辑结构，以及用户指定的打开、关闭、读、写等操作，还管理访问权限

考虑磁盘的物理磁道和扇区结构，逻辑访问必须转换成物理磁盘地址；磁盘存储空间和内存缓冲区的分配

I/O软件层次



- (1) 用户进程层执行输入输出系统调用，对I/O数据进行格式化，为假脱机输入/输出作准备
- (2) 独立于设备的软件实现设备的命名、设备的保护、成块处理、缓冲技术和设备分配
- (3) 设备驱动程序设置设备寄存器、检查设备的执行状态
- (4) 中断处理程序负责I/O完成时，唤醒设备驱动程序进程，进行中断处理
- (5) 硬件层实现物理I/O的操作

设备独立性（设备无关性）

用户编写的程序可以访问任意I/O设备，无需事先指定设备

好处：

设备分配时的灵活性
易于实现I/O重定向

从用户角度：用户在编制程序时，使用逻辑设备名，由系统实现从逻辑设备到物理设备（实际设备）的转换，并实施I/O操作

从系统角度：设计并实现**I/O软件时**，除了直接与设备打交道的低层软件之外，其他部分的软件不依赖于硬件

*I/O*相关技术

1.引入缓冲技术解决什么问题？

操作系统中最早引入的技术

→ 解决CPU与I/O设备之间速度的不匹配问题

凡是数据到达和离去速度不匹配的地方均可采用缓冲技术

→ 提高CPU与I/O设备之间的并行性

→ 减少了I/O设备对CPU的中断请求次数，放宽CPU对中断响应时间的要求

缓冲技术实现

► 缓冲区分类

硬缓冲：由硬件寄存器实现（例如：设备中设置的缓冲区）

软缓冲：在内存中开辟一个空间，用作缓冲区

► 缓冲区管理

单缓冲

双缓冲

缓冲池（多缓冲，循环缓冲）：统一管理多个缓冲区，采用有界缓冲区的生产者/消费者模型对缓冲池中的缓冲区进行循环使用

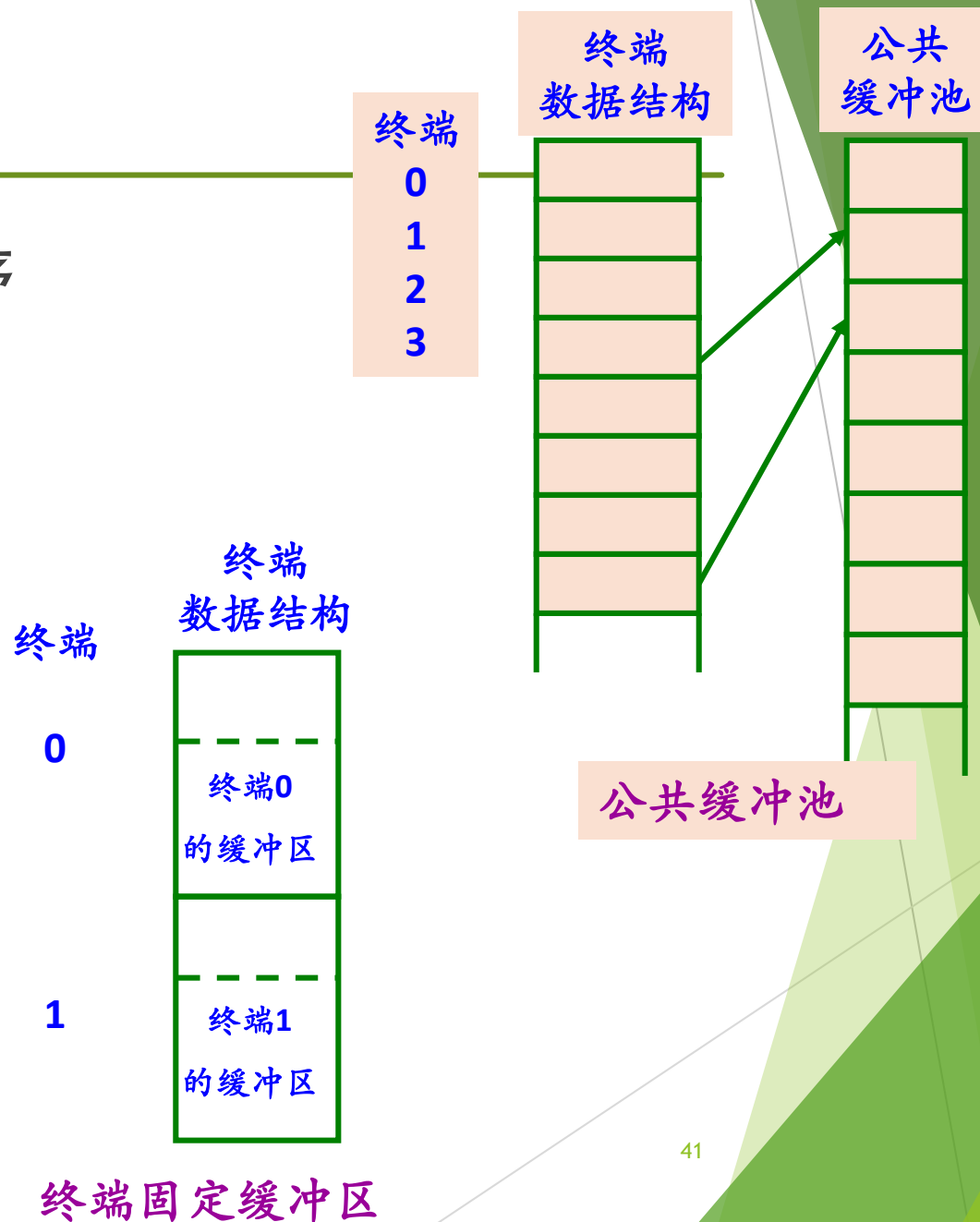
例子

▶ 终端输入软件中的键盘驱动程序

其任务之一：收集字符

两种常见的字符缓冲方法：

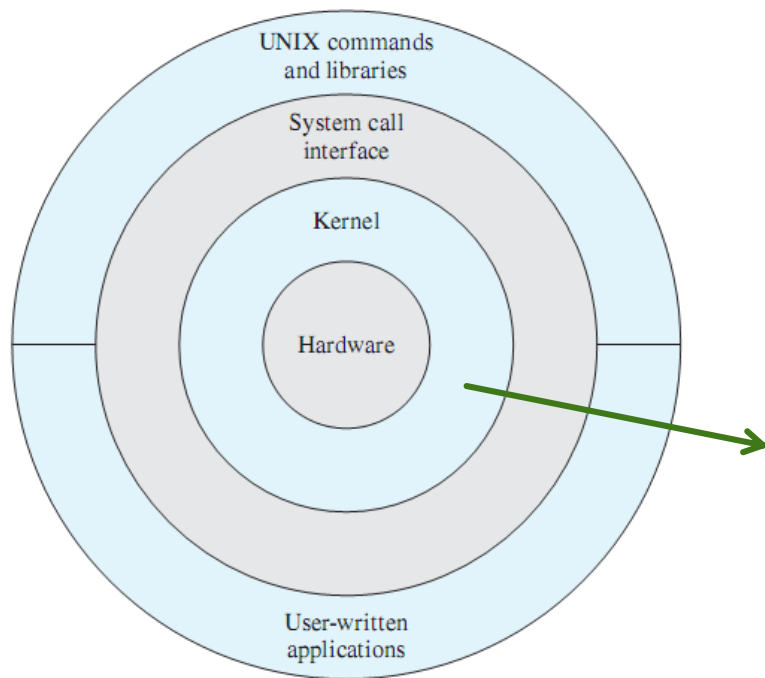
- ✓ 公共缓冲池（驱动程序中）
- ✓ 终端数据结构缓冲



UNIX System V 缓冲技术 (1/5)

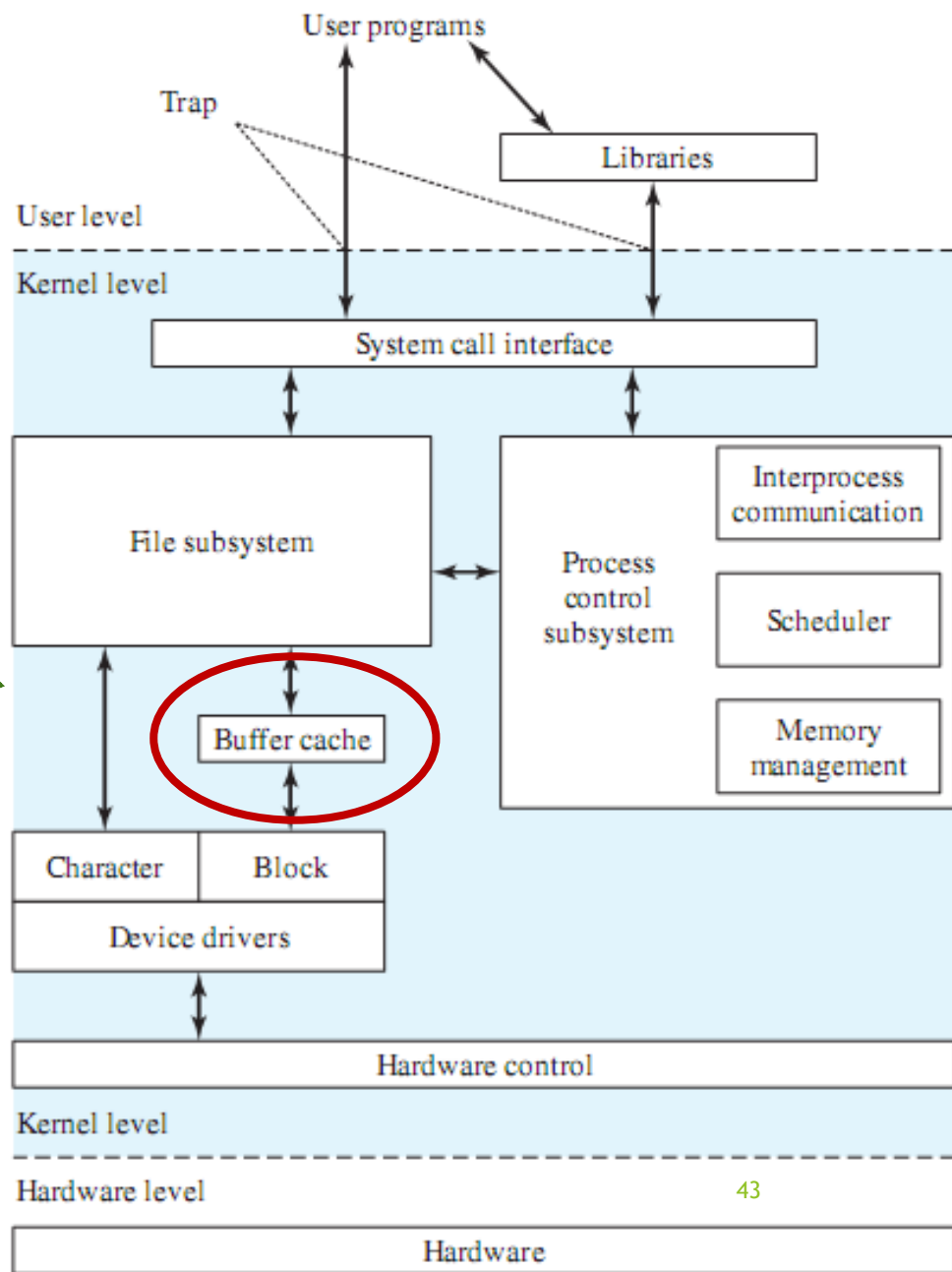
- ▶ 采用缓冲池技术，可平滑和加快信息在内存和磁盘之间的传输
- 缓冲区结合提前读和延迟写技术对具有重复性及阵发性I/O进程、提高I/O速度很有帮助
- 可以充分利用之前从磁盘读入、虽已传入用户区但仍在缓冲区的数据（尽可能减少磁盘I/O的次数，提高系统运行的速度）

UNIX操作系统



层次结构

内核结构



UNIX System V 缓冲技术 (2/5)

- ▶ 缓冲池：200个缓冲区(512字节或1024字节)

- ▶ 每个缓冲区由两部分组成：
缓冲控制块或缓冲首部 + 缓冲数据区

- ▶ 空闲缓冲区队列 (av链)

队列头部为bfreelist

- ▶ 设备缓冲队列 (b链)

链接所有分配给各类设备使用的缓冲区，按照散列方式组织

系统通过缓冲控制块来实现对缓冲区的管理

缓冲区用b双向链，假设有64个队列，每个队列首部有头标
设备为b_dev上的逻辑块b在散列队列的头标为：

$$i = (b_dev + b) \bmod 64$$

UNIX System V 缓冲技术 (3/5)

► 逻辑设备号和盘块号

分别标志出文件系统和数据所在的盘块号，是缓冲区的唯一标志

► 状态项

指明该缓冲区当前的状态：忙/闲、上锁/开锁、是否延迟写、数据有效性等

► 两组指针 (av和b)

用于对缓冲池的分配管理

设备号

盘块号

状态

指向缓冲数据区的指针

指向缓冲队列的后继指针 **b-back**

指向缓冲队列的前驱指针 **b-forw**

指向空闲队列的后继指针 **av-back**

指向空闲队列的前驱指针 **av-forw**

UNIX缓冲技术 (4/5)

每个缓冲区同时在av链和b链:

- ▶ 开始: 在空闲av链(缓冲区未被使用时)
- ▶ 开始IO请求: 在设备IO请求队列和设备b链
- ▶ IO完成: 在空闲av链和设备b链

UNIX System V 缓冲技术 (5/5)

系统对缓冲区的分配是
采用近似LRU算法

当进程想从指定的盘块读取数据时，系统根据盘块号在设备b链(散列队列)中查找，如找到缓冲区，则将该缓冲区状态标记为“忙”，并从空闲av队列中取下，然后完成从缓冲区到内存用户区的数据传送

- ✓ 如果在设备b链中未找到时，则从空闲av链队首摘取一个缓冲区，插入设备I/O请求队列；并从原设备b链中取下，插入由读入信息盘块号确定的新的设备b链中
- ✓ 当数据从磁盘块读入到缓冲区后，缓冲区从设备I/O请求队列取下；当系统完成从缓冲区到内存用户区的数据传送后，要把缓冲区释放，链入空闲av链队尾
- ✓ 当数据从磁盘块读入到缓冲区，并传送到内存用户区后，该缓冲区一直保留在原设备b链中，即它的数据一直有效。如它又要被使用，则又要从空闲av链中取下，使用完后插入到空闲av链队尾。如它一直未使用，则该缓冲区从空闲av链队尾慢慢升到队首，最后被重新分配，旧的盘块数据才被置换

2.设备管理有关的数据结构

- ▶ **描述设备、控制器等部件的表格：**系统中常常为每一个部件、每一台设备分别设置一张表格，常称为设备表或部件控制块。这类表格具体描述设备的类型、标识符、状态，以及当前使用者的进程标识符等
- ▶ **建立同类资源的队列：**系统为了方便对I/O设备的分配管理，通常在设备表的基础上通过指针将相同物理属性的设备连成队列（称设备队列）
- ▶ **面向进程I/O请求的动态数据结构：**每当进程发出I/O请求时，系统建立一张表格（称I/O请求包），将此次I/O请求的参数填入表中，同时也将该I/O有关的系统缓冲区地址等信息填入表中。I/O请求包随着I/O的完成而被删除
- ▶ **建立I/O队列：**如请求包队列

独占设备的分配

- ▶ 在申请设备时，如果设备空闲，就将其独占，不再允许其他进程申请使用，一直等到该设备被释放，才允许被其他进程申请使用
考虑效率问题，并避免由于不合理的分配策略造成死锁

静态分配：

在进程运行前，完成设备分配；运行结束时，收回设备

缺点：设备利用率低

动态分配：

在进程运行过程中，当用户提出设备要求时，进行分配，一旦停止使用立即收回

优点：效率好；缺点：分配策略不好时，产生死锁

分时式共享设备的分配

- ▶ 所谓分时式共享就是以一次I/O为单位分时使用设备，不同进程的I/O操作请求以排队方式分时地占用设备进行I/O
- ▶ 由于同时有多个进程同时访问，且访问频繁，就会影响整个设备使用效率，影响系统效率。因此要考虑多个访问请求到达时服务的顺序，使平均服务时间越短越好



3.设备驱动程序 (1/2)

- 与设备密切相关的代码放在设备驱动程序中，每个设备驱动程序处理一种设备类型
- 设备驱动程序的任务是接收来自与设备无关的上层软件的抽象请求，并执行这个请求
- 每一个控制器都设有一个或多个设备寄存器，用来存放向设备发送的命令和参数：设备驱动程序负责释放这些命令，并监督它们正确执行
- 当设备驱动程序释放一条或多条命令后，系统有两种处理方式：多数情况下，执行设备驱动程序的进程必须等待命令完成，这样，在命令开始执行后，它阻塞自己，直到中断处理时将它解除阻塞为止；而在其他情况下，命令执行不必延迟就很快完成

设备驱动程序 (2/2)

▶ 设备驱动程序与外界的接口

▶ 与操作系统的接口

为实现设备无关性，设备作为特殊文件处理。用户的I/O请求、对命令的合法性检查以及参数处理在文件系统中完成。在需要各种设备执行具体操作时，通过相应数据结构转入不同的设备驱动程序

▶ 与系统引导的接口

初始化，包括分配数据结构，建立设备的请求队列

▶ 与设备的接口

4.一种典型的实现方案：I/O进程 (1/2)

- ▶ **I/O进程**：专门处理系统中的I/O请求和I/O中断工作
- ▶ **I/O请求的进入**
 - ▶ 用户程序：调用send将I/O请求发送给I/O进程；调用block将自己阻塞，直到I/O任务完成后被唤醒
 - ▶ 系统：利用wakeup唤醒I/O进程，完成用户所要求的I/O处理
- ▶ **I/O中断的进入**
 - ▶ 当I/O中断发生时，内核中的中断处理程序发一条消息给I/O进程，由I/O进程负责判断并处理中断

一种典型的实现方案：I/O进程 (2/2)

► I/O进程

- 是系统进程，一般赋予最高优先级。一旦被唤醒，它可以很快抢占处理机投入运行
- I/O进程开始运行后，首先关闭中断，然后用receive去接收消息

两种情形：

- 没有消息，则开中断，将自己阻塞
- 有消息，则判断消息类型（I/O请求或I/O中断）

a. I/O请求

准备通道程序，发出启动I/O指令，继续判断有无消息

b. I/O中断，进一步判断正常或异常结束

正常：唤醒要求进行I/O操作的进程

异常：转入相应的错误处理程序

I/O性能问题

I/O性能问题

- ▶ 使CPU利用率尽可能不被I/O降低
- ▶ 使CPU尽可能摆脱I/O

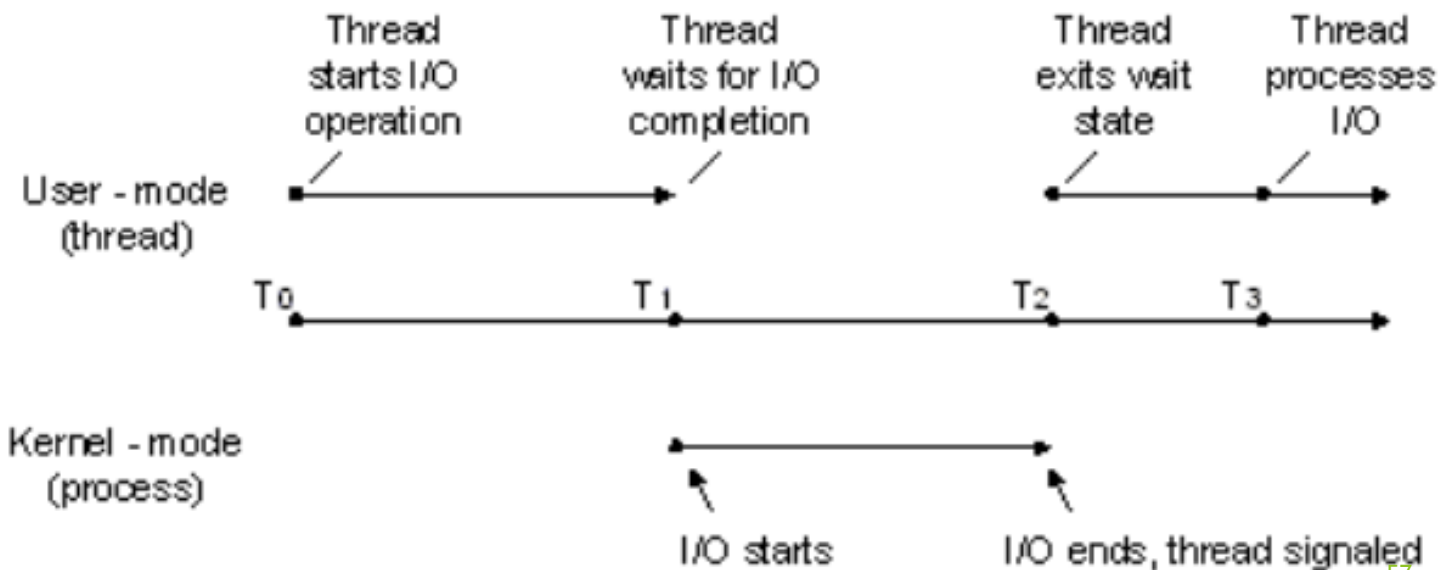
采取的措施

- ▶ 减少或缓解速度差距 → 缓冲技术
- ▶ 使CPU不等待I/O → 异步I/O
- ▶ 让CPU摆脱I/O操作 → DMA、通道

同步I/O

- ▶ 在I/O处理过程中，CPU处于空闲等待状态
- ▶ 而在处理数据的过程中，不能同时进行I/O操作

Synchronous I/O



异步传输

- ▶ Windows提供两种模式的I/O操作：
异步和同步

- ▶ 异步模式

用于优化应用程序的性能

- ▶ 通过异步I/O，应用程序可以启动一个I/O操作，然后在I/O请求执行的同时继续处理
- ▶ 基本思想：填充I/O操作间等待的CPU时间

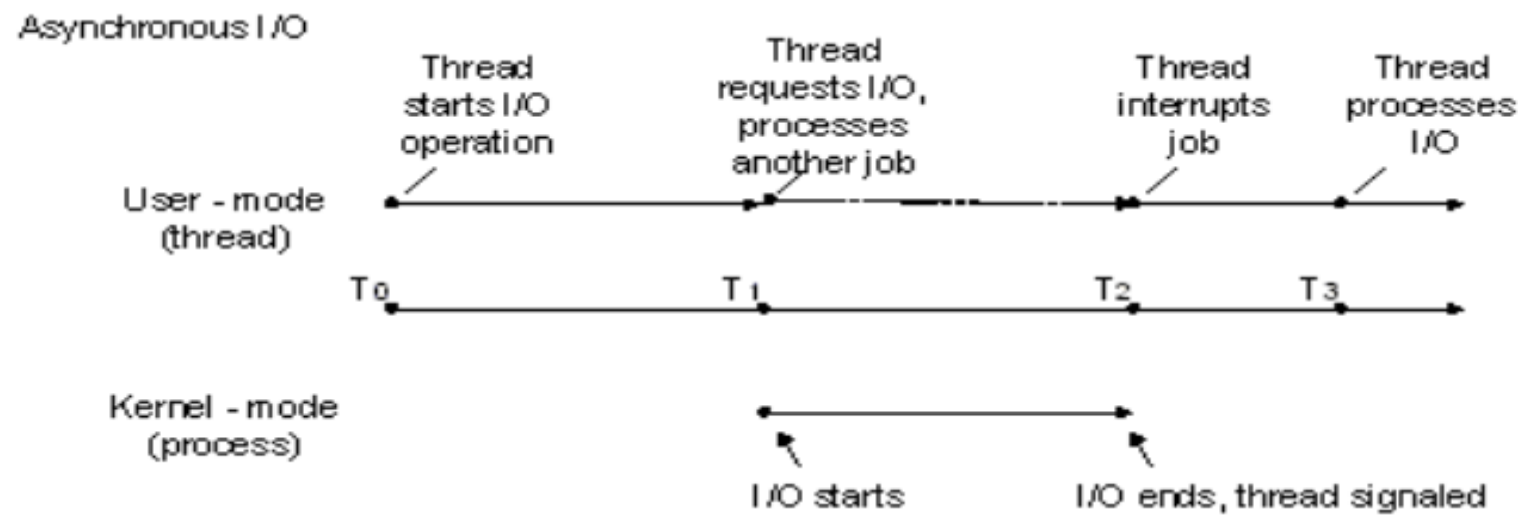
- ▶ 同步I/O

应用程序被阻塞直到I/O操作完成



阻塞I/O
非阻塞I/O

异步传输I/O的基本思想



- ◎ 系统实现
 - 通过切换到其他线程保证CPU利用率
 - 对少量数据的I/O操作会引入切换的开销
- ◎ 用户实现
 - 将访问控制分成两段进行
 - 发出读取指令后继续做其他操作
 - 当需要用读入的数据的时候，再使用wait命令等待其完成
 - 不引入线程切换，减少开销

重点小结

- ▶ I/O基本概念
 - ▶ 设备分类、设备管理的特点、设备无关性（设备独立性）
- ▶ I/O硬件组成
 - ▶ 设备控制器
 - ▶ 独立I/O端口、内存映射I/O
 - ▶ I/O控制方式
- ▶ I/O软件层次
 - ▶ 中断处理程序、设备驱动程序、I/O独立软件、用户级I/O软件
- ▶ I/O管理相关技术
- ▶ I/O性能问题

Many Thanks

The End